

Evaluación 3 Unidad 3: Árboles y Grafos

INF-213 Estructura de Datos

Rubén Sánchez

Diego Solís

Benjamín Vásquez

Joaquín Vásquez

Académico: PhD (c) Nicolás A. Reyes-Reyes

Fecha de entrega: 31 de junio de 2026

1 Introducción

Los grafos son una de las estructuras de datos no lineales más utilizadas para modelar problemas del mundo real en los que existen elementos interconectados, como redes de transporte, redes eléctricas o sistemas de rutas urbanas. A través de un grafo es posible representar tanto la posición de cada elemento (nodo) como el costo o distancia asociado a desplazarse entre ellos (arista), lo que permite aplicar algoritmos clásicos de optimización para resolver problemas concretos de conectividad y de recorrido. En este informe se aborda, por un lado, el problema del árbol de expansión mínima (MST) sobre un conjunto de 25 puntos de interés reales de la ciudad de Talca, comparando tres algoritmos distintos (Prim, Kruskal y Borůvka); y por otro lado, el problema del vendedor viajero (TSP) aplicado al diseño de un recorrido de bus de acercamiento de la Universidad Católica del Maule, resuelto mediante la metaheurística GRASP. El objetivo central es aplicar los TADs de grafos, conjuntos disjuntos y colas de prioridad estudiados en la asignatura, evaluando su correctitud y eficiencia frente a instancias de tamaño realista. Este tipo de problemas tiene una importancia práctica directa en el transporte y la logística, donde minimizar distancias recorridas se traduce en ahorros de tiempo, combustible y en la posibilidad de aumentar la frecuencia de los servicios ofrecidos.

2 Ejercicio 1: Árbol de Expansión Mínima

2.1 Modelado del problema

Para este ejercicio se seleccionó la ciudad de Talca, Región del Maule, debido a la buena disponibilidad de información geográfica de sus puntos de interés en Google Maps. Se identificaron 25 puntos de interés representativos de distintos sectores de la ciudad: plazas, universidades, hospitales, museos, centros comerciales, estaciones de transporte y zonas periféricas, entre otros. Cada punto de interés se modeló como un nodo del TAD **Grafo**, almacenando su nombre y sus coordenadas geográficas (latitud y longitud).

El grafo se construyó como un grafo no dirigido y completo: se agregó una arista entre cada par posible de los 25 nodos, lo que da un total de $\binom{25}{2} = 300$ aristas. El peso de cada arista se calculó mediante la función `distancia_euclidiana_aprox`, que aplica una proyección equirectangular sobre las coordenadas geográficas, corrigiendo la diferencia de longitud por el coseno de la latitud promedio entre ambos puntos y escalando el resultado por el radio de la Tierra (6.372,8 km). Esta aproximación es adecuada dado que todos los puntos de interés se encuentran a distancias relativamente pequeñas entre sí (dentro de la misma ciudad). Toda esta información se almacena en una matriz de adyacencia de 25×25 , que se imprime completa como parte de la salida del programa.

2.2 Algoritmos implementados

Se implementaron tres algoritmos distintos para obtener el árbol de expansión mínima (MST) del grafo construido:

- **Prim:** parte de un nodo inicial y hace crecer un único árbol agregando, en cada paso, la arista de menor peso que conecta el árbol actual con un nodo todavía no incluido.

Para encontrar eficientemente esa arista mínima se utilizó el TAD `MinHeap` (cola de prioridad implementada como heap binario en arreglo), que permite extraer el nodo de menor prioridad y disminuir prioridades en tiempo logarítmico.

- **Kruskal**: ordena todas las aristas del grafo de menor a mayor peso (usando `qsort`) y las recorre en ese orden, agregando cada una al MST solo si sus extremos no pertenecen todavía al mismo componente conexo, lo que se verifica mediante el TAD `ConjuntosDisjuntos` (Union-Find) con las optimizaciones de compresión de camino y unión por rango.
- **Borůvka**: a diferencia de los anteriores, trabaja por rondas. En cada ronda, cada componente conexo busca la arista más barata que lo conecta con un componente distinto, y todas esas aristas "ganadoras" se agregan simultáneamente, también apoyándose en `ConjuntosDisjuntos`. El número de componentes se reduce significativamente en cada ronda, por lo que el algoritmo converge en $O(\log N)$ rondas.

2.3 Pruebas de ejecución

Prueba 1: comparación de los tres algoritmos

Se ejecutaron los tres algoritmos sobre el mismo grafo de 25 nodos, partiendo Prim desde el nodo 0 (Plaza de Armas). Los resultados obtenidos en la ejecución real del programa fueron:

Algoritmo	Peso total del MST (km)
Prim	21.7309
Kruskal	21.7309
Borůvka	21.7309

Tabla 1: Comparación de pesos totales obtenidos por los tres algoritmos de MST.

Los tres algoritmos obtuvieron exactamente el mismo árbol de expansión mínima, con un peso total de 21,7309 km y 24 aristas (es decir, $N - 1$ aristas para $N = 25$ nodos), consistente con la definición de árbol de expansión. Algunas de las aristas más cortas encontradas corresponden a puntos cercanos del centro histórico, como Plaza de Armas – Centro Histórico (0,0929 km) o Centro Histórico – Museo O’Brien (0,0859 km), mientras que la arista más costosa del árbol conecta sectores periféricos, como Villa Cultural Huilquilemu – Camino a San Clemente (5,4124 km).

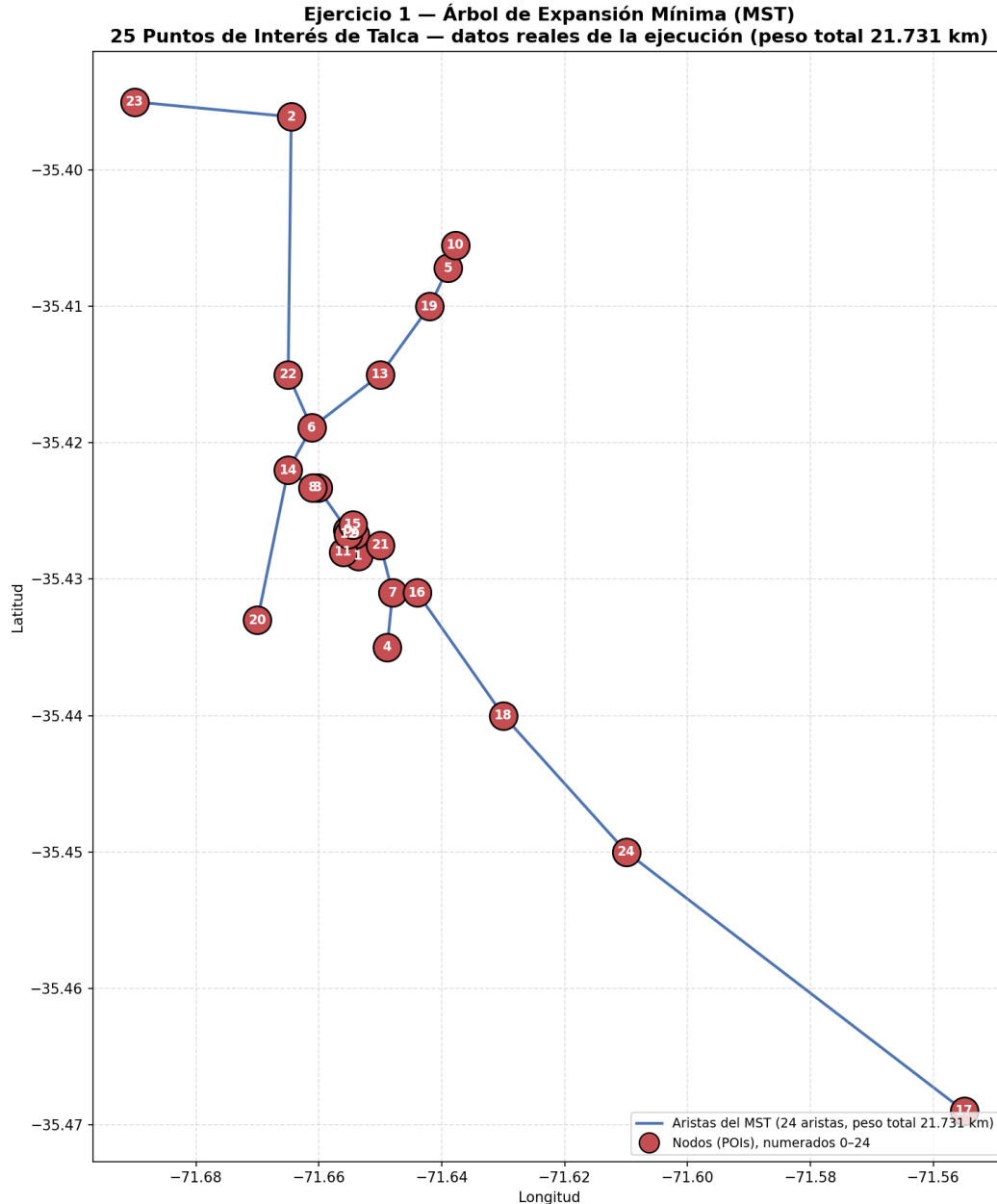


Figura 1: Árbol de expansión mínima obtenido sobre los 25 puntos de interés de Talca. Las coordenadas y las 24 aristas mostradas corresponden exactamente a la salida real del programa (archivo `resultado_ejercicio1.txt`); el peso total coincide con el reportado por los tres algoritmos (21,7309 km).

Prueba 2: Prim desde distintos nodos iniciales

Para verificar que el peso total del MST no depende del nodo desde el cual se inicia la construcción, se ejecutó el algoritmo de Prim partiendo desde tres nodos distintos: el nodo 0 (Plaza de Armas), el nodo 5 (Mall Plaza Maule) y el nodo 12 (Catedral de Talca).

Nodo inicial	Peso total del MST (km)
0 – Plaza de Armas	21.7309
5 – Mall Plaza Maule	21.7309
12 – Catedral de Talca	21.7309

Tabla 2: Peso total del MST obtenido con Prim desde distintos nodos de inicio.

La diferencia máxima entre los tres resultados fue de 0,000000 km, confirmando que el peso del MST es independiente del nodo de inicio elegido.

2.4 Análisis

Los resultados obtenidos confirman lo que predice la teoría de grafos: cuando un grafo es conexo y ponderado, el árbol de expansión mínima es único en cuanto a su peso total (aunque puede existir más de un conjunto de aristas que alcance ese mismo peso mínimo cuando hay pesos repetidos o muy similares entre sí). Los tres algoritmos implementados –Prim, Kruskal y Borůvka– parten de estrategias distintas (crecimiento de un único árbol, orden global de aristas, y fusión de componentes por rondas, respectivamente), pero al ser todos correctos desde el punto de vista de la teoría de optimización combinatoria, convergen al mismo resultado óptimo. La Prueba 2 refuerza esta conclusión desde otro ángulo: independientemente de cuál sea el nodo de partida, Prim siempre encuentra un árbol del mismo peso total, ya que el algoritmo garantiza optimalidad sin importar el punto de inicio, cambiando únicamente la forma en que se recorre el árbol pero no su costo final.

3 Ejercicio 2: Problema del Vendedor Viajero (TSP)

3.1 Modelado del problema

Para este ejercicio se modeló el recorrido del bus de acercamiento de la Universidad Católica del Maule (UCM) en la ciudad de Talca. Se definieron 5 zonas de la ciudad –Centro, Sur, Norte, Oriente y Poniente–, y dentro de cada zona se escogieron 10 ubicaciones reales (50 ubicaciones en total), correspondientes a lugares como universidades, parques, colegios, centros de salud y plazas de cada sector. La primera ubicación registrada en la zona Centro corresponde precisamente a la UCM, siendo el nodo de id 0 dentro del grafo.

Las ubicaciones dentro de cada zona se conectaron de forma completa (subgrafo K_{10}): cada una de las 10 ubicaciones de una zona es adyacente directamente a las otras 9 de la misma zona. Adicionalmente, se agregaron 5 conexiones entre ubicaciones de zonas distintas, enlazando sectores vecinos a través de puntos geográficamente cercanos entre ellos. Todas las distancias, tanto dentro de una zona como entre zonas distintas, se calcularon mediante la fórmula del **haversine**, que considera la curvatura de la Tierra y resulta más precisa que una aproximación euclidiana para este tipo de análisis de rutas.

Como el grafo resultante no es completo (solo existe conexión total dentro de cada zona y un número reducido de aristas entre zonas), no es posible conocer de antemano la distancia más corta entre dos ubicaciones de zonas distintas que no estén directamente conectadas.

Por este motivo se aplicó el algoritmo de **Floyd-Warshall**, que calcula la distancia del camino más corto entre todos los pares de nodos del grafo, considerando los posibles caminos a través de nodos intermedios. Esta matriz de distancias mínimas es la que efectivamente utiliza GRASP para evaluar y comparar los distintos circuitos candidatos.

3.2 Algoritmo GRASP

GRASP (*Greedy Randomized Adaptive Search Procedure*, o "Procedimiento de Búsqueda Voraz Adaptativo Aleatorizado") es una metaheurística que combina, en cada iteración, una fase de construcción voraz aleatorizada con una fase de búsqueda local, repitiendo este ciclo muchas veces para obtener una buena aproximación a la solución óptima.

Fase de construcción con RCL. En cada paso de la construcción del circuito, en lugar de elegir siempre el nodo no visitado más cercano (estrategia puramente voraz), se construye una *Lista de Candidatos Restringida* (RCL) que incluye a todos los nodos no visitados cuya distancia al último nodo agregado se encuentre dentro de un umbral. Dicho umbral se calcula como $\text{umbral} = d_{\min} + \alpha \cdot (d_{\max} - d_{\min})$, donde $d_{\min}$ y $d_{\max}$ son, respectivamente, la menor y la mayor distancia entre el nodo actual y los nodos aún no visitados. Finalmente, se elige aleatoriamente uno de los nodos de la RCL para continuar el circuito. El parámetro $\alpha \in [0, 1]$ controla el balance entre *explotación* y *exploración*: con $\alpha = 0$, solo el nodo más cercano puede entrar a la RCL (comportamiento voraz puro); con $\alpha = 1$, todos los nodos no visitados entran a la RCL (selección esencialmente aleatoria).

Búsqueda local 2-opt. Una vez construido un circuito completo, se aplica el movimiento 2-opt: se prueban pares de aristas del circuito y se evalúa si invertir el segmento de ruta entre ellas reduce la distancia total. El nodo de inicio (la UCM) se mantiene siempre fijo en su posición, ya que el recorrido debe comenzar y terminar en ese punto. El proceso se repite hasta que ninguna inversión adicional logra mejorar el circuito, alcanzando así un óptimo local.

3.3 Pruebas de ejecución

Prueba 1: GRASP con $\alpha = 0,3$ y 200 iteraciones

Se ejecutó GRASP con $\alpha = 0,3$ durante 200 iteraciones sobre el grafo de 50 ubicaciones. El mejor circuito encontrado por el programa obtuvo una distancia total de **56,2558 km**, visitando las 50 ubicaciones de las 5 zonas y retornando finalmente a la UCM. El recorrido agrupa de forma natural las ubicaciones cercanas entre sí (dentro de una misma zona) antes de desplazarse hacia la siguiente zona, lo cual es coherente con la estructura del grafo, donde las conexiones intra-zona son mucho más numerosas que las conexiones inter-zona.

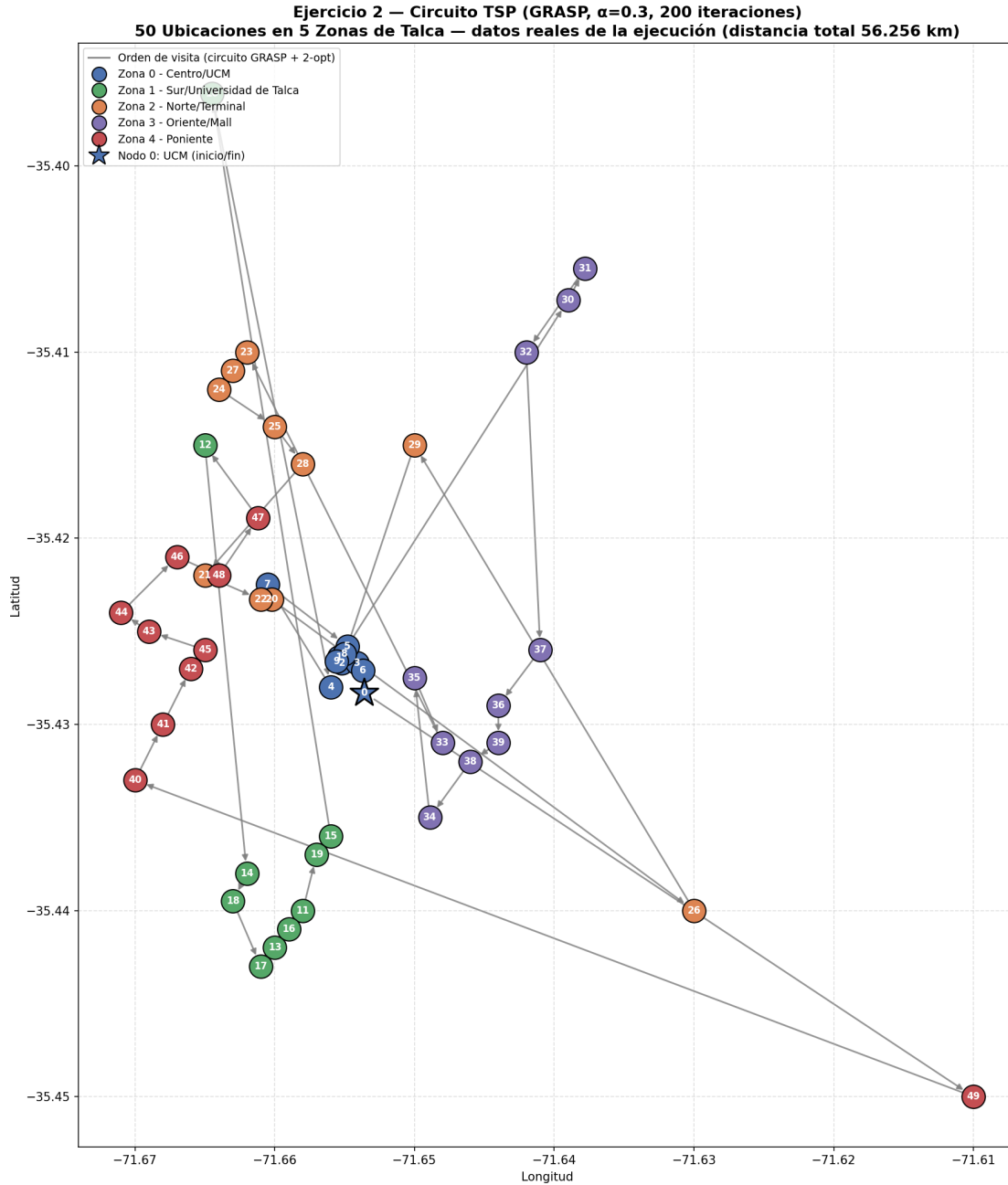


Figura 2: Circuito TSP obtenido por GRASP ($\alpha = 0,3$, 200 iteraciones) sobre las 50 ubicaciones de las 5 zonas de Talca. Las flechas indican el orden exacto de visita reportado por el programa (archivo `resultado_ejercicio2.txt`); el nodo 0 (estrella) corresponde a la UCM, punto de inicio y de retorno del circuito. La distancia total (56,2558 km) coincide con la reportada en la ejecución real.

Prueba 2: comparación de distintos valores de alpha

Se ejecutó GRASP con cuatro valores distintos de α (0,0, 0,3, 0,7 y 1,0), cada uno con 200 iteraciones, obteniendo los siguientes resultados:

Valor de α	Distancia total (km)
0,0	57,2978
0,3	56,2558
0,7	56,2558
1,0	56,2558

Tabla 3: Distancia total del mejor circuito GRASP para distintos valores de α (200 iteraciones cada uno).

3.4 Análisis y recomendación

A diferencia de lo que suele esperarse de forma intuitiva –que un comportamiento totalmente voraz ($\alpha = 0$) entregue siempre el mejor resultado–, en esta instancia particular del problema se observó el efecto contrario: $\alpha = 0$ obtuvo la *peor* distancia total (57,2978 km), mientras que $\alpha = 0,3, 0,7$ y $1,0$ convergieron de manera consistente a un resultado mejor (56,2558 km). Este comportamiento, lejos de ser un error, ilustra una propiedad importante de las meta-heurísticas tipo GRASP: cuando $\alpha = 0$, la RCL contiene prácticamente un único candidato en cada paso, por lo que la fase de construcción se vuelve *determinista*. Esto significa que las 200 iteraciones generan exactamente el mismo circuito inicial, y la búsqueda local 2-opt dispone de un solo punto de partida desde el cual intentar mejorar la solución, quedando atrapada en su óptimo local correspondiente. En cambio, con $\alpha > 0$ cada iteración construye un circuito inicial distinto, lo que le da a la fase de búsqueda local múltiples puntos de partida diferentes desde los cuales explorar el espacio de soluciones, aumentando la probabilidad de escapar de óptimos locales mediocres y de encontrar un circuito de menor distancia total.

Para el caso del bus de acercamiento de la UCM, esta observación tiene una implicancia práctica relevante: no conviene utilizar GRASP en su variante completamente voraz, ya que pierde la capacidad exploratoria que es justamente la fuente de su buen desempeño práctico frente a una heurística voraz simple. Se recomienda utilizar un valor de α moderado (en torno a 0,3), que mantiene una construcción mayoritariamente guiada por la cercanía geográfica –favoreciendo rutas razonables y evitando saltos excesivos entre ubicaciones lejanas– pero conservando suficiente diversidad entre iteraciones para que la búsqueda local 2-opt pueda explorar distintos óptimos locales y converger a una solución de buena calidad, tal como lo muestran los resultados obtenidos en las pruebas de ejecución.

4 Conclusiones

El desarrollo de esta evaluación permitió aplicar de manera práctica los TADs de estructuras no lineales estudiados en el curso –grafos, conjuntos disjuntos y colas de prioridad– sobre problemas con datos geográficos reales. La implementación de Prim, Kruskal y Borůvka evidenció que, aunque cada algoritmo recorre el espacio de soluciones de una forma distinta, todos son capaces de alcanzar el árbol de expansión mínimo óptimo cuando el grafo es conexo, reforzando la importancia de elegir la estructura auxiliar adecuada (heap para Prim, conjuntos disjuntos para Kruskal y Borůvka) según las necesidades de cada algoritmo. Por su parte, la resolución del TSP mediante GRASP mostró que la recursividad y las estructuras

de datos eficientes no bastan por sí solas para resolver problemas NP-difíciles de gran escala, y que metaheurísticas que combinan aleatoriedad controlada con búsqueda local pueden encontrar soluciones de buena calidad en tiempos razonables. En conjunto, este trabajo deja en evidencia que la elección de la estructura de datos y del algoritmo correcto no es una decisión menor, sino que determina directamente la correctitud, eficiencia y calidad de la solución obtenida frente a problemas de optimización que surgen habitualmente en contextos reales de transporte y logística urbana.

5 Anexo

El código fuente completo que resuelve ambos ejercicios se adjunta como un archivo comprimido (`proyecto_grafos.zip`) junto con este informe, siguiendo la estructura de proyecto del IDE CLion. El proyecto incluye los siguientes módulos principales:

- `salida.h` / `salida.c`: impresión simultánea en consola y archivo.
- `arbol.h` / `arbol.c`: TAD de conjuntos disjuntos (Union-Find).
- `arbol_binario.h` / `arbol_binario.c`: TAD min-heap (cola de prioridad).
- `distancias.h` / `distancias.c`: cálculo de distancias geográficas (euclidiana aproximada y haversine).
- `grafo.h` / `grafo.c`: TAD grafo no dirigido y ponderado, mediante matriz de adyacencia.
- `caminos_minimos.h` / `caminos_minimos.c`: algoritmo de Floyd-Warshall.
- `grasp_tsp.h` / `grasp_tsp.c`: algoritmo GRASP (RCL + búsqueda local 2-opt).
- `mst.h` / `mst.c`: algoritmos de Prim, Kruskal y Borůvka.
- `main_ejercicio1.c` y `main_ejercicio2.c`: programas principales de cada ejercicio.

A continuación se muestra, a modo de referencia, un fragmento del TAD de conjuntos disjuntos utilizado tanto por Kruskal como por Borůvka:

```
1 int cd_encontrar(ConjuntosDisjuntos *cd, int x) {
2     if (cd->padre[x] == x) {
3         return x;
4     }
5     /* Compresion de camino: x apunta directamente a la raiz */
6     cd->padre[x] = cd_encontrar(cd, cd->padre[x]);
7     return cd->padre[x];
8 }
9
10 void cd_unir(ConjuntosDisjuntos *cd, int x, int y) {
11     int raiz_x = cd_encontrar(cd, x);
12     int raiz_y = cd_encontrar(cd, y);
13 }
```

```

14     if (raiz_x == raiz_y) {
15         return; /* Ya pertenecen al mismo conjunto */
16     }
17     if (cd->rango[raiz_x] < cd->rango[raiz_y]) {
18         cd->padre[raiz_x] = raiz_y;
19     } else if (cd->rango[raiz_x] > cd->rango[raiz_y]) {
20         cd->padre[raiz_y] = raiz_x;
21     } else {
22         cd->padre[raiz_y] = raiz_x;
23         cd->rango[raiz_x]++;
24     }
25 }

```

Listing 1: Fragmento de arbol.c: union por rango con compresion de camino